

食材管理 — 实施方案

2026-07-23 · 厨房小秘书项目

摘要

开发一个名为「食材管理」的移动端优先单页 Web 应用。交付物为 index.html + init.sql。使用原生 HTML/CSS/JS，后端 Supabase REST API，AI 集成 DeepSeek Chat。核心功能：多用户/多仓库库存管理 + “说人话就能管库存” 的 AI 文本录入。

第一步：SQL 建表脚本 (init.sql)

7 张表

表	关键字段	说明
accounts	id, username, password, role	用户账号，role 为 user/admin
warehouses	id, name, owner_id, created_at	仓库
warehouse_members	id, warehouse_id, user_id, role	成员关系，role 为 owner/staff
locations	id, warehouse_id, name, type	位置，type 为 常温/冷藏/冷冻
inventory	id, warehouse_id, name, qty, location_id, warning	库存项，warning 存部分移库提示
recipes	id, warehouse_id, name, ingredients, steps	菜谱 (ingredients 为 jsonb)
logs	id, warehouse_id, user_id, action, detail, log_type	操作日志

约束

- inventory 上 (warehouse_id, name) 不设唯一约束（支持追加序号方案）
- 所有表 warehouse_id 外键 + 索引

2 个 RPC 函数

- delete_warehouse_cascade(warehouse_id) — 级联删除仓库及所有关联数据
- delete_user_cascade(user_id) — 级联删除用户及 warehouse_members 记录

RLS 策略 (针对 anon key)

- accounts: SELECT 仅自己 (admin 全部) / INSERT 开放注册 / UPDATE 仅自己 / DELETE 仅 admin
- warehouses: SELECT 通过 warehouse_members 或 admin / INSERT 任意用户 / UPDATE owner/admin / DELETE 走 RPC
- warehouse_members: SELECT 仓库成员或 admin / INSERT/UPDATE/DELETE owner/admin
- locations/inventory/recipes: SELECT 仓库成员 / INSERT/UPDATE/DELETE 仓库成员
- logs: SELECT 仓库成员 / INSERT 仓库成员 (只追加)

第二步：实现 index.html

单 HTML 文件，→ <style> → <script> 顺序。

CSS

- CSS 变量：番茄红/暖橙主色、米白背景、琥珀黄警告、正红
- 移动端优先，max-width: 480px 居中布局
- 标题字体用站酷系列 CDN 字体，正文系统无衬线
- 动效：状态圆点呼吸动画、模态框底部滑入、toast 顶部滑入、过期行脉冲

数据层

- 统一 supabaseGet/Patch/Post/Delete 函数，携带 apikey + Bearer 头
- 全局数据对象 window.__data: { inventory, locations, recipes, logs, members }
- 数据刷新函数 refreshData(warehouseId): 并发拉取 5 个数据集
- 懒加载守卫：页面切换时数据为空则自动拉取
- 所有写操作：先写云端 → 成功后更新 __data → 最后渲染

路由状态机

- 三个页面视图：login, register, main，用 JS 控制显示/隐藏
- 页面加载时检查 ks_auto_login + ks_current_user 决定跳转

认证模块

- 注册：前端校验 (2-20 字符、密码≥6位) → 查重 → 明文写入 accounts → 跳转登录
- 登录：查 accounts → 比对密码 → 写 localStorage → 加载仓库列表 → 进入主页
- 退出：清除 ks_current_user + ks_auto_login
- 修改密码：旧密码校验 → 更新 accounts

仓库模块

- 创建：弹窗输入名称 → 写入 warehouses + warehouse_members(owner) → 初始化 4 个默认位置
- 切换：点击顶部仓库名 → 弹出仓库列表 (根据 role 确定数据源)
- 删除：owner/admin 可见 → 二次确认 → 调 RPC 级联删除
- 成员管理：成员列表 + 邀请员工 + 移除员工 + 指定新老板 (owner/admin 可见)

系统管理员面板

- 双击顶部角色标签 (仅 admin) → 用户管理、删除用户、Excel 批量导入、查看全部仓库、一键清除本地缓存

库存模块

- 列表渲染：按入库时间倒序，受位置筛选条件影响
- 折叠/展开：默认显示前 N 条，ks_show_all 持久化状态
- 手动新增：查云端同名最大序号 → 追加序号写入
- 删除：推入 ks_undo_queue，5 分钟内可撤回
- 快速改位置：点选式弹出位置列表
- 过期预警：常温>24h黄档/>48h红档，冷藏冷冻>7天黄档

- 部分移库警告：数量右上方小问号，点击弹窗提示

AI 文本录入 (核心)

- 底部固定输入框 (最多50字) + 发送按钮
- 提交流程: 文本清洗 → 拼装 System Prompt → 请求 DeepSeek (15s超时) → 解析 JSON → 分发
- add_stock: 循环 items 写入库存
- use_stock: 匹配库存条, 多条时弹出选择界面让用户选
- move_stock: 更新位置; 部分移走则原记录加 warning 标记
- new_location: 弹窗询问是否新增该位置
- ask_qty: 气泡反问用户数量, 等待补充输入
- chat: 直接气泡展示回复
- 非法 JSON 重试 1 次, 仍失败则提示"AI 未能理解"
- AI 处理期间禁用输入框, 防止重复提交
- 每次成功操作写入日志

菜谱模块

- 新增/删除菜谱弹窗 (名称 + 食材 JSON + 步骤), 卡片式列表, 点击展开

日志模块

- 同时写云端 (含 warehouse_id) + 追加 ks_logs_cache 本地缓存
- 时间倒序渲染, 删除类日志 5 分钟内显示「撤回」按钮

边界与异常

- 所有 DOM 插入前转义防 XSS
- Supabase 超时 10s / DeepSeek 超时 15s
- 请求失败 toast "网络异常, 请重试"
- 云端不可达时显示错误页 + 重试按钮
- 每次操作后静默校验用户仓库权限 (被移出时自动切换)
- 撤回队列每次渲染时清理过期项

测试计划

- SQL 脚本验证: 在 Supabase SQL Editor 中执行 init.sql, 确认无报错
- RPC 验证: 手动调用 delete_warehouse_cascade 和 delete_user_cascade 确认级联生效
- 手动流程测试: 注册 → 登录 → 创建仓库 → 增删改库存 → AI 录入 → 切仓库 → 成员管理 → 退出
- Admin 面板测试: 用户管理、Excel 导入、查看全部仓库
- AI 测试: 输入各种自然语言指令, 确认 action 分发正确
- 异常测试: 断开网络后操作、非法 JSON 返回、超时场景

假设与默认选择

- 手动新增食材走查云端最大序号 → 追加序号逻辑, AI 录入也走相同逻辑
- 并发竞态兜底: 查序号和写入之间若冲突, 捕获唯一约束异常后重试一次

- 密码明文存储（用户确认）
- ks_saved_accounts 明文存储（用户确认）
- locations 的 type 字段在创建时由用户从 “常温/冷藏/冷冻” 三选一
- 交付物为 index.html + init.sql